

Graphify, LLM Wiki e second brain con Claude Code e Obsidian

Executive summary

I tre oggetti della richiesta non sono equivalenti, ma si incastrano molto bene se separati per livello architetturale. **Graphify** è soprattutto un estrattore e interrogatore di conoscenza strutturale su codebase e corpora tecnici multimodali: parte da file, ricava un grafo interrogabile, e lo espone come HTML, JSON, wiki markdown, vault Obsidian o server MCP. **LLM Wiki secondo Karpathy** è invece un *pattern* di knowledge management persistente: un LLM mantiene nel tempo una wiki markdown composta da fonti grezze, pagine sintetiche e uno schema operativo, così da accumulare conoscenza invece di “riscoprirla” a ogni query. **Claude Code + Obsidian** non è un metodo alternativo ai due precedenti, ma l’ambiente operativo più naturale per farli vivere: Obsidian fornisce il vault markdown locale e navigabile, Claude Code porta memoria, skill, hook e automazioni agentiche. ¹

La scelta pratica dipende dallo scopo. Per **sviluppo software** e comprensione architetturale di repository complessi, Graphify è la scelta più forte. Per **ricerca accademica** e costruzione di una base di conoscenza che componga progressivamente paper, articoli e note, il pattern LLM Wiki è spesso più naturale. Per **gestione personale della conoscenza** e workflow quotidiani, la combinazione più equilibrata è Obsidian come base, con Claude Code come layer agentic e, se serve, una implementazione LLM Wiki o Graphify come estensioni specialistiche. Per **team** piccoli o medi, la soluzione più robusta tende a essere ibrida: repo/vault in markdown e Git, Obsidian per interfaccia umana, Claude Code per le routine, Graphify per la mappa strutturale dei progetti. ²

Un punto importante: Graphify e LLM Wiki **non sono concorrenti perfetti**. Graphify può già esportare una wiki markdown e un vault Obsidian, mentre Karpathy descrive esplicitamente Obsidian come l’ambiente in cui osservare e navigare la wiki mantenuta dal modello. La differenza chiave è quindi il **modello mentale dei dati**: Graphify pensa in nodi/archi/comunità; LLM Wiki pensa in pagine/pagine collegate/index/log; Claude Code + Obsidian pensa in file markdown, regole persistenti e tooling operativo. ³

Metodo e chiarimenti terminologici

Nelle fonti primarie il cognome corretto è **Karpathy**, non “Karpaty”; in questo report uso quindi la forma corretta. Inoltre, la richiesta cita “**Code Code**”: nelle fonti ufficiali, nei plugin Obsidian e nell’ecosistema pratico rilevante compare in modo consistente **Claude Code**, cioè il tool agentic di Anthropic. Per coerenza con le fonti e con il caso d’uso descritto, tratto quindi “Code Code” come probabile refuso per **Claude Code**. Dove un elemento non emerge chiaramente dalle fonti ufficiali — per esempio un repository GitLab canonico di Graphify o una “licenza” univoca del *metodo* LLM Wiki — lo segnalo esplicitamente come **non specificato**. ⁴

Tra le risorse ufficiali in italiano effettivamente disponibili ho trovato: la **README it-IT di Graphify** e la **panoramica di Claude Code in italiano**. Per Obsidian e per il pattern LLM Wiki di Karpathy, le fonti ufficiali più ricche restano prevalentemente in inglese. ⁵

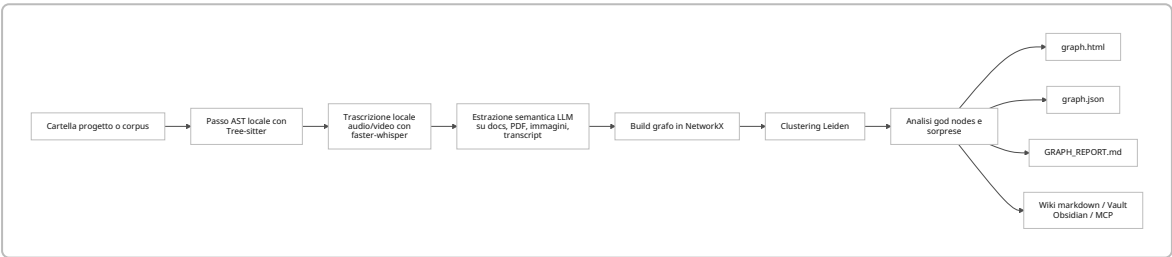
Graphify

Graphify è un progetto open source che trasforma cartelle di codice, documentazione, PDF, immagini e video in un **knowledge graph interrogabile** destinato ad assistenti di coding come Claude Code, Codex, OpenCode, Cursor, Gemini CLI e altri. La repository ufficiale indicata dalle fonti del progetto è `safishamsi/graphify` **su GitHub**; la README italiana aggiunge in modo esplicito che **“l’unico repository ufficiale è safishamsi/graphify”**. Un repository GitLab ufficiale non risulta indicato nelle fonti ufficiali consultate, quindi va trattato come **non specificato**. Il pacchetto PyPI ufficiale è `graphifyy` con doppia “y”, mentre altri pacchetti `graphify*` non sono dichiarati affiliati. La licenza corrente del repository è **MIT**. ⁶

Aspetto	Evidenza
Repository GitHub ufficiale	<code>safishamsi/graphify</code> ⁷
Repository GitLab ufficiale	non specificato nelle fonti ufficiali consultate ⁸
Package ufficiale	<code>graphifyy</code> su PyPI; CLI <code>graphify</code> ⁹
Licenza	MIT ⁹
Runtime minimo	Python 3.10+ ⁹

Tecnicamente, Graphify lavora in una pipeline a passaggi separati. Il primo passaggio usa **Tree-sitter** per estrarre struttura di codice in locale, senza LLM: classi, funzioni, import, call graph, commenti inline; per SQL estrae anche tabelle, viste, chiavi esterne e relazioni di join in modo deterministico. Il secondo passaggio trascrive video e audio con **faster-whisper** in locale. Il terzo passaggio manda documenti, paper, immagini e trascrizioni a subagenti LLM in parallelo, che restituiscono frammenti JSON poi fusi in un unico grafo. Infine il grafo viene costruito in **NetworkX**, clusterizzato con **Leiden**, analizzato per “god nodes” e connessioni sorprendenti, ed esportato come `graph.html`, `graph.json` e `GRAPH_REPORT.md`. Il progetto dichiara anche etichette di confidenza sugli archi: `EXTRACTED`, `INFERRED` e `AMBIGUOUS`. ¹⁰

La pipeline ufficiale di Graphify può essere resa così. ¹¹



Sul piano dell’architettura software, la documentazione ufficiale riassume i moduli principali come `detect`, `extract`, `build`, `cluster`, `analyze`, `report`, `export`, con moduli di supporto come `cache.py`, `security.py`, `watch.py`, `ingest.py` e `serve.py`. Nella snapshot del `pyproject.toml` consultata, le dipendenze core comprendono **NetworkX** e le famiglie **Tree-sitter** per numerosi linguaggi; gli extra opzionali includono `mcp`, `neo4j`, `pdf` con `pypdf` e `markdownify`, `watch` con `watchdog`, `svg` con `matplotlib`, `leiden` con `graspologic`, `office`, `video`, `kimi` e `sql`. Questo rende Graphify molto più vicino a una *toolchain multimodale per code intelligence* che a un semplice RAG. ¹²

Gli esempi d'uso ufficiali sono concreti e abbastanza maturi. Il comando base è `/graphify .`; poi si possono eseguire query (`/graphify query`), percorsi minimi tra nodi (`/graphify path`), spiegazioni (`/graphify explain`), ingest di URL o video (`/graphify add`), rebuild automatici con `graphify hook install`, interrogazione diretta del grafo da terminale con `graphify query`, oppure avvio di un server MCP con `python -m graphify.serve graphify-out/graph.json`. La README documenta anche export verso **Obsidian**, **wiki markdown**, **GraphML**, **Neo4j** e **MCP**, oltre a una modalità di team setup in cui `graphify-out/` viene committato nel repository e un merge driver evita conflitti su `graph.json`. ¹³

Dal punto di vista privacy e sicurezza, Graphify distingue bene tra ciò che resta locale e ciò che dipende dal backend LLM. Il codice viene analizzato in locale con Tree-sitter; la security policy afferma che durante l'analisi del grafo il tool **non effettua chiamate di rete**, salvo il sottocomando `ingest` quando l'utente chiede esplicitamente di recuperare un URL, e che l'MCP server lavora via **stdio**. La landing page ufficiale aggiunge che non c'è telemetria, che il progetto usa il model/API key già configurato dall'assistente e che, per il codice sorgente, invia descrizioni semantiche invece del sorgente grezzo. La stessa security policy documenta mitigazioni esplicite contro SSRF, oversized downloads, path traversal, XSS e prompt injection nei label dei nodi. ¹⁴

Lo **stato manutentivo** appare molto attivo, ma anche sotto pressione. Nella verifica effettuata il repository mostrava circa **51k stelle**, **5.5k fork**, **511 commit**, **106 issue** e **170 pull request** aperte; la UI GitHub mostrava aggiornamenti recentissimi. Ci sono segnali di manutenzione reale — changelog, sicurezza, tests, PR comunitarie — ma anche backlog significativo. Alcuni punti rilevanti emersi dalle issue/PR consultate: richiesta di parsing PDF locale per minimizzare esposizione dati, bug sulla gestione di `.graphifyignore` in `--watch`, limiti di efficienza su codebase molto grandi, e un issue di maggio 2026 che chiedeva l'aggiunta di una licenza esplicita; oggi il repository mostra una licenza MIT, quindi quel blocco di governance sembra essere stato affrontato. La PR #804, ancora draft, aggrega hardening, supporto bash, packaging e estrazione immagini da PDF; la PR #534 affronta invece una regressione sugli hook per Codex. In aggiunta, il sito ufficiale `graphify.net` mostrava un contatore "3.7k+ GitHub Stars", molto inferiore rispetto alla UI GitHub, il che suggerisce che almeno alcuni indicatori del sito marketing fossero in ritardo rispetto allo stato reale del repository. ¹⁵

In termini pratici, Graphify è ideale quando servono **comprensione strutturale**, **navigazione di grandi codebase**, **bridge tra codice e documentazione**, **query a basso budget token**, o una vista comune di progetto per il team. È meno ideale quando il problema principale non è la struttura del software ma la **sintesi lenta e progressiva della conoscenza** su un tema: in quel caso il pattern LLM Wiki resta più naturale. I limiti principali da tenere presenti sono quattro: dipendenza eventuale da backend LLM per contenuto non-codice, complessità di setup superiore a una wiki markdown semplice, possibile backlog manutentivo del progetto, e benchmark di compressione/token reduction prodotti dal progetto stesso e non da validazione indipendente. ¹⁶

LLM Wiki secondo Karpathy

Il riferimento primario del "metodo Karpathy" è una **GitHub Gist** di Andrej Karpathy intitolata "**LLM Wiki**". Non è presentata come prodotto finito né come repository software canonico: Karpathy la definisce un "**idea file**" da copiare nel proprio agente LLM, così che l'agente e l'utente costruiscano insieme l'implementazione concreta. Questo punto è fondamentale: **LLM Wiki non è, nelle fonti primarie, un software unico**, ma un *pattern di progettazione* per basi di conoscenza personali o di team gestite da LLM. Di conseguenza, per il "repository ufficiale" del metodo il dato corretto è: **GitHub Gist di karpathy/llm-wiki**; **repository GitLab ufficiale non specificato**. Una licenza univoca del *metodo* non è specificata; le implementazioni comunitarie hanno licenze proprie. ¹⁷

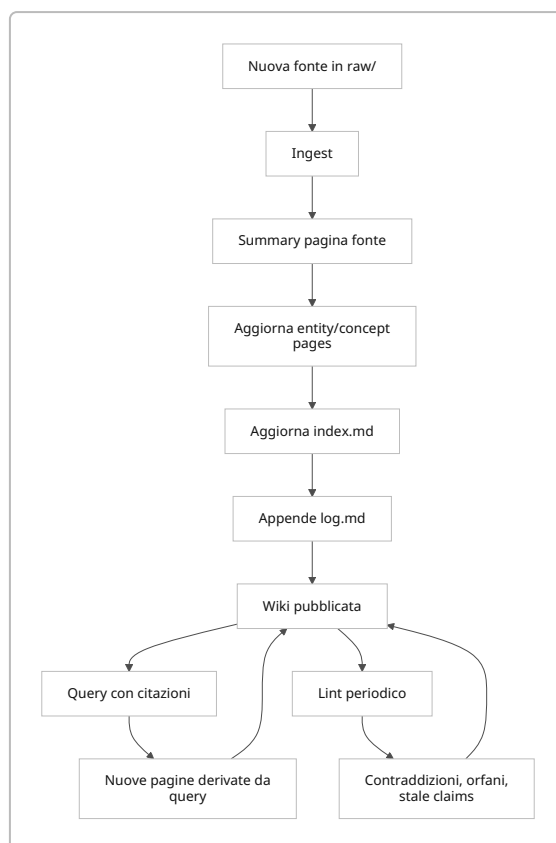
Aspetto	Evidenza
Riferimento primario	Gist GitHub <code>karpthy/llm-wiki</code> ¹⁸
Repository software canonico	non specificato ; la fonte primaria è una Gist, non un pacchetto/repo applicativo ¹⁸
Repository GitLab ufficiale	non specificato ¹⁸
Licenza del “metodo”	non specificato ; le implementazioni hanno licenze proprie ¹⁷

La tesi di Karpathy è una critica precisa al RAG tradizionale. Nel RAG classico, l'LLM recupera frammenti rilevanti al momento della domanda e ricostruisce la risposta ogni volta; nella LLM Wiki, invece, l'LLM **legge una nuova fonte, la sintetizza una volta, aggiorna pagine esistenti, segnala contraddizioni e mantiene una wiki interlinkata persistente**. La conoscenza quindi non viene “riderivata” a ogni query: viene *compilata* una volta e poi raffinata progressivamente. Karpathy esplicita perfino la metafora operativa: Obsidian come IDE, il modello come programmatore, la wiki come codebase. ¹⁸

L'architettura concettuale ufficiale del pattern è molto pulita e ha **tre layer**. Il primo sono le **raw sources**, cioè le fonti curate dall'utente e non modificate dal modello. Il secondo è la **wiki**, cioè le pagine markdown interconnesse, scritte e mantenute dall'LLM. Il terzo è lo **schema**, tipicamente un `CLAUDE.md` o `AGENTS.md`, che descrive convenzioni, struttura e workflow. Le operazioni fondamentali sono tre: **ingest**, **query** e **lint**. Durante l'ingest, una nuova fonte può aggiornare 10-15 pagine; in query, il modello legge prima l'indice e poi le pagine rilevanti, producendo risposte citate che a loro volta possono essere archiviate nella wiki; nel lint, il modello cerca contraddizioni, pagine orfane, claims obsoleti, buchi informativi e link mancanti. Due file sono centrali: `index.md` per la navigazione contenutistica e `log.md` come ledger cronologico append-only. Karpathy dice anche che questa strategia funziona sorprendentemente bene a **scala moderata** — circa 100 fonti e centinaia di pagine — senza dover introdurre subito infrastruttura embedding-based. ¹⁸

Sul piano operativo, il pattern suggerisce anche tooling opzionale ma molto concreto: **Obsidian Web Clipper** per trasformare articoli in markdown, **qmd** o strumenti simili se la wiki cresce e serve una ricerca locale più robusta, **Obsidian graph view** per vedere hub/orfani, **Dataview** per generare tabelle dinamiche da frontmatter e **Marp** per produrre slide decks a partire dal contenuto della wiki. Inoltre, Karpathy insiste sul fatto che la wiki sia semplicemente **un repository Git di file markdown**, quindi con version history, branching e collaborazione via Git “gratis”. ¹⁸

Il workflow tipico del pattern può essere sintetizzato così. ¹⁸



Poiché il pattern è deliberatamente astratto, le implementazioni comunitarie sono parte importante della valutazione. Tra quelle più significative nelle fonti consultate: `kytmanov/obsidian-llm-wiki-local`, che propone una pipeline locale con Ollama e vault Obsidian, con `olw run` come orchestratore `ingest → compile → lint → approve`, test offline e niente telemetria; `sametbrr/llm-wiki-manager`, che implementa il pattern come skill per Claude Code con struttura `raw/`, `wiki/`, `CLAUDE.md`, script idempotenti e supporto multi-wiki routing; `flsteven87/llm-wiki-mcp`, che lo incapsula come MCP server + skill Claude Code, dichiarandosi ancora alpha e richiedendo Python 3.11+; e `TrueH00HA/LLM-Wiki-Skilled`, che mette l'accento sull'uso di skill agentiche per ridurre deviazioni comportamentali. Le licenze osservate sono MIT per diversi progetti e Apache-2.0 per `LLM-Wiki-Skilled`. ¹⁹

Per ancorare il pattern a una letteratura più stabile, è utile distinguerlo da alcuni vicini accademici. **RAG** in senso stretto, come formalizzato da Lewis et al. nel 2020, combina memoria parametrica e memoria non parametrica con recupero di passaggi a query-time; **GraphRAG** di Microsoft aggiunge estrazione strutturata e summarization gerarchica su grafi per corpora privati; **MemGPT** propone una gerarchia di memorie per agenti LLM; il riferimento culturale più remoto evocato dallo stesso Karpathy è il **Memex** di Vannevar Bush. Nessuno di questi lavori è "il metodo Karpathy", ma tutti aiutano a collocarlo: il metodo Karpathy è soprattutto un **pattern di knowledge compilation persistente in markdown**, non una tecnica di retrieval specifica o un algoritmo peer-reviewed. Ad oggi, nelle fonti ufficiali consultate, **non risulta un paper accademico firmato da Karpathy dedicato a LLM Wiki**. ²⁰

I vantaggi del pattern sono notevoli per ricerca e knowledge management: persistenza, accumulo, citazioni file-to-file, facilità di audit grazie a `log.md`, costo cognitivo inferiore rispetto al mantenimento manuale di una wiki, e forte compatibilità con Git e Obsidian. I limiti sono altrettanto chiari: assenza di specifica canonica, qualità fortemente dipendente dallo schema e dal modello usato, rischio di drift o sovra-produzione di pagine, revisione umana necessaria sulle contraddizioni, e scalabilità non illimitata

senza introdurre strumenti di ricerca aggiuntivi. Se si cerca una soluzione “chiavi in mano”, il pattern puro può risultare troppo aperto; se invece si cerca un sistema flessibile, trasparente e *LLM-native*, è una proposta molto potente. ²¹

Second brain con Claude Code e Obsidian

Il concetto di **second brain** è stato popolarizzato da **Tiago Forte** come metodo per organizzare la vita digitale e sbloccare creatività e produttività. In pratica, nel contesto di questa richiesta, “second brain” significa una base di conoscenza esterna, durabile, ricercabile e riusabile, costruita su file e strumenti che aiutano sia la memoria personale sia l'azione quotidiana. In un setup tecnico moderno, Obsidian fornisce la base dati locale in markdown; Claude Code aggiunge lo strato agentico che legge, scrive, automatizza e ricorda regole operative. ²²

Claude Code è descritto da Anthropic come uno strumento di coding agentico disponibile in terminale, IDE, app desktop e browser, capace di leggere la codebase, modificare file, eseguire comandi e integrarsi con gli strumenti di sviluppo. La memoria persistente passa da due meccanismi: `CLAUDE.md` per regole e istruzioni scritte dall'utente, e **auto memory** per note che Claude salva da solo, entrambe caricate a inizio sessione. Inoltre, i **hook** permettono comandi shell automatici in momenti specifici del lifecycle, e le **skills** impacchettano workflow riusabili. Questo lo rende molto adatto a operare sopra un vault o sopra una wiki in modo deterministico e ripetibile, non solo conversazionale. ²³

Obsidian archivia le note come **file Markdown plain-text in un vault sul file system locale**, aggiorna automaticamente il vault quando i file cambiano dall'esterno e può essere sincronizzato con Obsidian Sync, Git e vari servizi terzi. Questa caratteristica è esattamente ciò che lo rende una base eccellente per un second brain assistito da agenti: i contenuti restano file leggibili, versionabili e manipolabili da CLI, plugin e script. A livello di prodotto, Obsidian è oggi **gratuito per tutti gli usi**, inclusi quelli commerciali; i servizi opzionali includono ad esempio **Sync** a 4 USD/mese per utente con billing annuale o 5 USD/mese con billing mensile. ²⁴

La forma più semplice del setup è: **vault Obsidian + Claude Code lanciato nella root del vault + `CLAUDE.md` con convenzioni del knowledge base**. Claude Code può allora usare il vault come working directory, mantenere regole persistenti, automatizzare template, validazioni e import, mentre Obsidian resta l'interfaccia umana per navigare link, graph view, frontmatter, basi dati e annotazioni. Questo schema è coerente sia con la memoria nativa di Claude Code sia con il pattern LLM Wiki di Karpathy. ²⁵

Le integrazioni oggi esistenti nell'ecosistema Obsidian sono già sufficienti per comporre un second brain operativo senza costruirsi tutto da zero. La tabella seguente riassume le più rilevanti emerse nelle fonti ufficiali e semi-ufficiali consultate.

Integrazione	Ruolo pratico	Evidenza
Claudian	Incorpora Claude Code/Codex nel vault; il vault diventa working directory dell'agente con lettura/scrittura file, bash, multi-step workflow e supporto MCP.	²⁶
Claude Panel	Aggiunge una chat laterale alimentata dal <code>claude</code> CLI come subprocess, con session resume, mention automatiche e MCP project-level.	²⁷

Integrazione	Ruolo pratico	Evidenza
Open in Claude Code	Apri la directory della nota o la root del vault in Claude Code dalla command palette/ribbon e sincronizza <code>.claude/settings.json</code> .	28
Vault as MCP	Espone il vault come server MCP locale, con privacy locale e senza telemetria; utile per collegare LLM esterni al vault.	29
Obsidian Git	Version control e backup Git dentro il vault.	30
Local Backup	Backup locali schedulati o su evento.	31
Templater	Template dinamici e JavaScript nei template.	32
QuickAdd	Cattura rapida, macro, template e workflow di inserimento.	33
Omnisearch	Ricerca avanzata su note, PDF e OCR.	34
Smart Connections	Collegamenti semantici con indicizzazione locale del vault.	35
Dataview	Query e viste su metadata/frontmatter del vault.	36
Zotero Integration	Import di citazioni, bibliografie, note e annotazioni PDF da Zotero.	37

Per **sincronizzazione e versioning**, le opzioni più solide sono tre. La prima è **Git**, favorito dal fatto che il vault è plain text e dall'esistenza di Obsidian Git. La seconda è **Obsidian Sync**, che sincronizza a livello file, mantiene una copia locale, supporta version history ed è **end-to-end encrypted** per default; Obsidian fornisce anche una modalità **Headless** per usare Sync da riga di comando, con casi d'uso espliciti di backup remoti, automazioni pianificate e accesso di tool agentici a un vault senza concedere loro pieno accesso al computer. La terza sono i servizi di sync file-based, che però Obsidian tratta con maggior cautela: la documentazione raccomanda di non tenere il vault attivo dentro cartelle di sync di terze parti sullo stesso device quando si usa anche Obsidian Sync, e ricorda che un servizio di sync non sostituisce un vero backup. 38

Per **team**, Obsidian è utilizzabile, ma con limiti che vanno compresi prima di adottarlo come knowledge platform condivisa. Nelle shared vault via Obsidian Sync, tutti i collaboratori devono avere una subscription Sync attiva; le autorizzazioni non sono ancora granulari; non c'è collaborative live editing sullo stesso file; le modifiche si propagano via sync e i conflitti possono essere uniti e ispezionati via version history; la documentazione indica inoltre un massimo di 20 collaboratori per shared vault. Per team tecnici questo significa che Obsidian funziona bene come **knowledge hub asincrono**, meno come editor collaborativo in tempo reale alla Google Docs. 39

Sul fronte rischi, il primo non è tecnico ma operativo: **plugin sprawl**. Obsidian ha superato i 4.000 plugin e 120 milioni di download complessivi; nel 2026 ha introdotto un nuovo directory/community site con review automatiche continue, scorecard di sicurezza e, progressivamente, disclosure sulle capacità di rete/filesystem/clipboard dei plugin. È una buona notizia per l'ecosistema, ma il corollario è che in un second brain "AI-heavy" conviene mantenere un set di plugin il più piccolo e comprensibile possibile. Il secondo rischio è economico: Claude Code è incluso nei piani Claude Pro/Max, ma la documentazione spiega anche che l'uso via API resta fatturato a consumo token; i costi dipendono quindi parecchio da modello, codebase, parallelismo e automazioni. 40

Confronto e raccomandazioni pratiche

La comparazione più utile non è “quale vince?”, ma **a quale livello del problema agisce ciascuno**. La tabella seguente è una sintesi ragionata delle fonti ufficiali e delle repository primarie consultate. Le valutazioni qualitative su facilità d'uso, scalabilità e privacy sono inferenze motivate dalle caratteristiche documentate, non punteggi ufficiali dei progetti. ⁴¹

Asse	Graphify	LLM Wiki secondo Karpathy	Claude Code + Obsidian
Che cos'è	Tool open source che costruisce un knowledge graph interrogabile da codice e corpus tecnici	Pattern per wiki markdown persistente mantenuta dall'LLM	Stack operativo: agente + vault markdown + plugin/ sync
Unità di conoscenza	Nodi, archi, comunità, shortest path	Pagine, index, log, cross-link	File markdown, regole <code>CLAUDE.md</code> , automazioni
Workflow dominante	Estrarre struttura e interrogare il grafo	Ingerire fonti e far crescere una wiki persistente	Lavorare e automatizzare dentro il vault
Facilità d'uso	Media	Media-bassa	Media
Scalabilità	Medio-alta per codebase/corpora tecnici	Media; oltre una certa taglia richiede search/ tooling extra	Media-alta per uso personale o team piccoli
Privacy	Medio-alta: AST locale, ma contenuti non-codice possono coinvolgere backend LLM	Variabile: può essere molto alta con implementazioni locali	Variabile: vault locale forte, ma plugin/sync/ Claude incidono
Costo	Software base gratuito; costo LLM/backend variabile	non specificato come pattern; dipende dall'implementazione	Obsidian base gratuito; Sync opzionale; Claude piano/API a parte
Integrazioni	Molto alte: MCP, Obsidian, wiki, Neo4j, GraphML	Alte se costruite bene, ma non standardizzate	Molto alte grazie a plugin, MCP, Git, Headless Sync
Best fit	Sviluppo software, code intelligence, architecture mapping	Ricerca, studio profondo, knowledge management persistente	Second brain personale, runbook, note di progetto, hub operativo

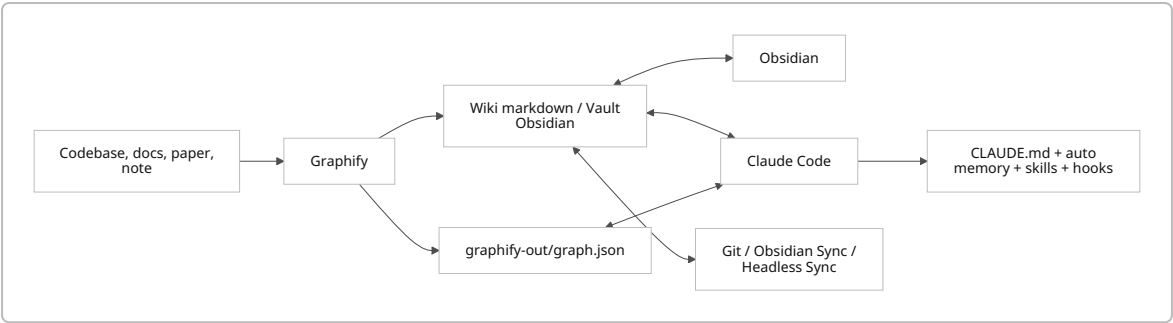
Se dovessi raccomandare **una scelta secca per scenario**, la mia risposta sarebbe questa. Per **ricerca accademica individuale**, scegliere prima **Obsidian + Zotero Integration**, poi aggiungere un'implementazione **LLM Wiki** se vuoi sintesi cumulativa, soprattutto se lavori per settimane o mesi sullo stesso dominio. Per **sviluppo software**, scegliere **Graphify** quando contano struttura, dipendenze, percorsi e comprensione cross-file; affiancare **Claude Code + Obsidian** se vuoi anche memoria di decisioni, runbook, note di debug e playbook di team. Per **gestione personale della**

conoscenza, partire da **Obsidian** e aggiungere Claude Code solo quando serve automazione reale; Graphify qui è spesso sovradimensionato salvo corpus tecnici o code-heavy. Per **team**, preferire una soluzione controllata e file-first: **vault/repo markdown + Git o Obsidian Sync + regole** `CLAUDE.md` + **sottoinsieme minimo di plugin**, eventualmente con Graphify per i progetti software. ⁴²

Esigenza	Scelta consigliata	Motivo principale
Ricerca accademica	LLM Wiki su Obsidian; opzionalmente implementazione locale <code>obsidian-llm-wiki-local</code>	Trasforma paper e note in una base di conoscenza cumulativa, auditabile e citata ⁴³
Sviluppo software	Graphify	Massimizza comprensione strutturale di codebase e relazioni tra codice, SQL, docs e PR ⁴⁴
PKM personale	Obsidian + Claude Code leggero	Miglior compromesso tra controllo locale, semplicità e automazione graduale ⁴⁵
Team engineering	Obsidian/Git/Claude Code + Graphify mirato	Wiki/runbook condivisi + mappa del codice; file-first e più governabile di una piattaforma chiusa ⁴⁶
Massima privacy locale	<code>obsidian-llm-wiki-local</code> o Vault-as-MCP + modelli locali	Fonti e query restano sulla macchina o su infrastruttura locale ⁴⁷

La raccomandazione più forte del report, però, è **ibrida**: usare **Graphify** per mappare in profondità i repository e i corpora tecnici; usare **Obsidian** come datastore e interfaccia umana durabile; usare **Claude Code** come layer agentico; usare il **pattern LLM Wiki** per la parte di sintesi persistente che deve crescere nel tempo. È la combinazione che massimizza trasparenza, portabilità dei dati e potenza operativa, senza costringerti a scegliere tra grafo e wiki. ⁴⁸

Uno stack ibrido ragionevole si presenta così. ⁴⁹



Dettagli rimasti non specificati nelle fonti consultate. Un repository GitLab ufficiale per Graphify non è indicato; una repository software “ufficiale” per il metodo LLM Wiki di Karpathy non esiste nelle fonti primarie oltre alla Gist; una licenza univoca del metodo LLM Wiki non è specificata; un paper accademico firmato da Karpathy su LLM Wiki non risulta dalle fonti ufficiali esaminate. In questi casi conviene trattare il pattern come concetto aperto e valutare attentamente la qualità dell’implementazione concreta che si decide di adottare. ⁵⁰

1 3 6 7 9 13 15 41 46 48 49 <https://github.com/safishamsi/graphify>
<https://github.com/safishamsi/graphify>

2 4 17 18 20 21 43 [llm-wiki · GitHub](https://github.com/safishamsi/graphify)
<https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>

5 8 50 [graphify/docs/translations/README.it-IT.md at v4 · safishamsi/graphify · GitHub](https://github.com/safishamsi/graphify/blob/v4/docs/translations/README.it-IT.md)
<https://github.com/safishamsi/graphify/blob/v4/docs/translations/README.it-IT.md>

10 11 44 <https://github.com/safishamsi/graphify/blob/v8/docs/how-it-works.md>
<https://github.com/safishamsi/graphify/blob/v8/docs/how-it-works.md>

12 16 [Graphify — Open-Source Knowledge Graph Skill for AI Coding Assistants](https://graphify.net/)
<https://graphify.net/>

14 [graphify/SECURITY.md at v8 · safishamsi/graphify · GitHub](https://github.com/safishamsi/graphify/blob/v8/SECURITY.md)
<https://github.com/safishamsi/graphify/blob/v8/SECURITY.md>

19 47 <https://github.com/kytmanov/obsidian-llm-wiki-local>
<https://github.com/kytmanov/obsidian-llm-wiki-local>

22 <https://www.buildingasecondbrain.com/>
<https://www.buildingasecondbrain.com/>

23 <https://docs.anthropic.com/it/docs/claude-code/overview>
<https://docs.anthropic.com/it/docs/claude-code/overview>

24 38 45 <https://obsidian.md/help/data-storage>
<https://obsidian.md/help/data-storage>

25 <https://docs.anthropic.com/en/docs/claude-code/memory>
<https://docs.anthropic.com/en/docs/claude-code/memory>

26 <https://community.obsidian.md/plugins/realclaudian>
<https://community.obsidian.md/plugins/realclaudian>

27 <https://community.obsidian.md/plugins/claude-panel-ryukuhub>
<https://community.obsidian.md/plugins/claude-panel-ryukuhub>

28 <https://community.obsidian.md/plugins/open-in-claude-code>
<https://community.obsidian.md/plugins/open-in-claude-code>

29 <https://community.obsidian.md/plugins/vault-as-mcp>
<https://community.obsidian.md/plugins/vault-as-mcp>

30 <https://community.obsidian.md/plugins/obsidian-git>
<https://community.obsidian.md/plugins/obsidian-git>

31 <https://community.obsidian.md/plugins/local-backup>
<https://community.obsidian.md/plugins/local-backup>

32 <https://community.obsidian.md/plugins/templater-obsidian>
<https://community.obsidian.md/plugins/templater-obsidian>

33 <https://community.obsidian.md/plugins/quickadd>
<https://community.obsidian.md/plugins/quickadd>

34 <https://community.obsidian.md/plugins/omnisearch>
<https://community.obsidian.md/plugins/omnisearch>

35 <https://community.obsidian.md/plugins/smart-connections>
<https://community.obsidian.md/plugins/smart-connections>

36 <https://github.com/blacksmithgu/obsidian-dataview>

<https://github.com/blacksmithgu/obsidian-dataview>

37 42 <https://community.obsidian.md/plugins/obsidian-zotero-desktop-connector>

<https://community.obsidian.md/plugins/obsidian-zotero-desktop-connector>

39 <https://obsidian.md/help/teams/sync>

<https://obsidian.md/help/teams/sync>

40 <https://obsidian.md/blog/future-of-plugins/>

<https://obsidian.md/blog/future-of-plugins/>